Pipelining: Tests

- Imagine we have a non-pipelined processor running at 10MHz and want to run a program with 1000 instructions.
  a) How much time would it take to execute the program?

- Assuming ideal conditions (balanced pipeline stages, no hazards), how much time would it take to execute the same program in:
  b) A 10-stage pipeline?
  c) A 100-stage pipeline?

- Looking at those results, it seems clear that increasing pipeline depth should increase the execution speed of a processor. Why do you think that processor designers (e.g., Intel, see below) have not only stopped increasing pipeline length but, in fact, reduced it?

Pentium III – Coppermine (1999)	*10-stage pipeline*
Pentium IV – NetBurst (2000)	*20-stage pipeline*
Pentium Prescott (2004)	*31-stage pipeline*
Core i7 9xx – Bloomfield (2008)	*24-stage pipeline*
Core i7 5Yxx – Broadwell (2014)	*19-stage pipeline*
Core i7 77XX – Kaby Lake (2017)	*~20-stage pipeline*

Imagine we have a non-pipelined processor running at 10 MHz and want to run a program with 1000 instructions.

a) How much time (in microseconds) would it take to execute the program?

- Assuming one instruction per cycle, and that 1 cycle = $\frac{1}{10*10^6} = 0.1\ \mu\text{sec}$
- Execution time can be calculated as: $1000 \times\ 0.1\ \mu\text{sec}$ = **100 µsec**

Assuming ideal pipelining (balanced stages, no hazards), how much time would it take to execute the same 1000-instructions program in:

a) A **10-stage** pipeline?

- Assuming ideal pipelining gives a 10x speed-up in clock frequency, therefore $1 \text{ cycle} = \frac{1}{10 \times 10^7} = 0.01 \mu\text{sec}$
- Assuming an ideal throughput of 1 completed instruction per cycle, and an initial latency of 9 cycles to fill up the pipeline
- Program will take: (1000 + 9) × 0.01 µsec = **10.09 µsec**

Assuming ideal pipelining (balanced stages, no hazards), how much time would it take to execute the same 1000-instructions program in:

b) A **100-stage** pipeline?

- 1 cycle now takes 1ns, and 99 cycles are initially required to fill up the pipeline
- Program will take: $(1000 + 99) \times 0.001$ µsec = **1.099 µsec**

Key Takeaway

- A 100-stage pipeline cuts down on execution time by roughly 100x
  - With an N-stage pipeline, the expected speed-up is N
  - Why did we get a speed-up of less than 10 (actually, 9.9) and 100 (90.9), respectively?
  - Because the pipeline, although balanced by construction, needs time to get to regime!

An N-stage pipeline exhibits an Nx increase in throughput over a corresponding single-cycle implementation provided

- The pipeline is balanced
- The pipeline is full
- Register overhead is omitted

Limits to Pipelining

- Area and power overhead
  - More complex datapath (registers, forwarding lanes) and control
  - Clock distribution network overhead
- Increased sensitivity to data and control hazards
  - Diminishing returns on throughput speedup
- Higher penalties for data hazards and branch mispredictions
- Increased sensitivity to register overhead
- Problems in generating a well-formed high-speed clock signal